

CSE587: Networks for AI/ML Systems
Endsem (2 hrs) [Total: 62 points]
Dec 6, 2025 [2 hrs]

Note: Answer ANY SIX questions from Q.2. to Q.8. The rest are COMPULSORY questions.

Q.1. Suppose you are training a large model using **Data Parallelism** across **8 GPUs (one GPU per server node)**. Each GPU computes **gradients of size 128 MB**, which must be averaged across all GPUs before proceeding to the next iteration. **[2+2+2+2+2=10]**

(a) Identify which collective operation is best suited for this task and explain why other alternatives are insufficient.

(b) Describe the phases of the collective communication algorithm (that you picked in (a)) using the example mentioned in the problem statement.

(c) Calculate the **total amount of data** sent by each GPU if the **ring-based algorithm** is used.

(d) If link bandwidth = 200Gbps, **estimate the communication time** per GPU for one DP iteration if the **ring-based algorithm** is used.

(e) Estimate the communication time if **tree-based in-network compute** is implemented at the switch (all 8 GPUs connect to the switch) with link bandwidth = 200Gbps. Assume infinite compute and I/O capacity at the switch.

Ans.

(a) AllReduce

AllReduce combines the reduction (sum) and distribution (broadcast of the reduced result) into a single, communication-optimized primitive that avoids an explicit central root, reduces total data movement, and enables efficient algorithms (e.g., ring/pipelined) that overlap communication and reduce network contention compared to a separate Reduce followed by Broadcast.

(b) AllReduce Ring-based algorithm comprises two phases:

1. Reduce-Scatter phase (first phase):

The full tensor is partitioned into N equal chunks (i.e., 8 here). Over N-1 steps each GPU sends one chunk to its neighbor and receives one chunk from its other neighbor, while performing a local element-wise reduce (sum) with the received chunk. After this phase each GPU holds one disjoint chunk containing the reduced result for that chunk.

2. All-Gather phase (second phase):

The N reduced chunks are distributed so every GPU ends up with the full reduced tensor. Over another N-1 steps each GPU sends one chunk to its neighbor and receives one chunk, accumulating chunks until it has the complete tensor.

(c) Given:

- Number of GPUs (or nodes): $N = 8$
- Data size (gradient tensor): $S = 128\text{MB}$

AllReduce = two phases:

CSE587: Networks for AI/ML Systems

Endsem (2 hrs) [Total: 62 points]

Dec 6, 2025 [2 hrs]

1. Reduce-Scatter phase: Gradients are summed (reduced) and scattered so each GPU ends up with one reduced chunk.

2. All-Gather phase: The reduced chunks are shared back so every GPU has the full result.

Each phase involves (N – 1) communication steps.

We split the tensor of size S into **N equal chunks**.

So each chunk = $S/N = 16\text{MB}$

At each step:

- Every GPU **sends** one chunk and **receives** one chunk simultaneously.
- So in each step, a GPU transfers (send + receive) one chunk = S/N of data.

But these two happen **concurrently** on full-duplex links, so the **effective data per step** = S/N

So total data transferred per GPU per phase = $2 \times (N-1) \times (S/N)$ bytes.

$$= 2 * 7 * 16\text{MB} = 224\text{MB}$$

(d) Each GPU communicates over a link of bandwidth B.

So communication time $T = \text{total data} / \text{bandwidth}$

$$T = (2 \times (N-1) \times (S/N) \text{ bytes}) / B$$

$$T = (2 * 7 * (128\text{MB} / 8)) / 200\text{Gbps}$$

$$T = (2 * 3 * (128\text{M} / 8) * 8) / 200\text{Gbps} = 8.96 \text{ ms}$$

(e) In the case of tree-based in-network compute, all GPUs share their gradients with the switch. The switch performs the reduce operation, and shares the aggregated results back to the GPUs.

Since the switch has infinite compute and I/O capacity, each GPU can share all the gradients at once, i.e., 128MB.

Since all GPUs will share the gradients in parallel, the time to share the gradients with the switch = $128\text{MB}/200\text{Gbps} = 5.12 \text{ ms}$

The time to return the aggregated value from the switch to the GPUs = $128\text{MB}/200\text{Gbps} = 5.12 \text{ ms}$

Total communication time = $5.12 + 5.12 = 10.24\text{ms}$

Q.2. How would you design a connection-establishment and teardown mechanism that supports high scalability (e.g., thousands to millions of endpoints/connections) while keeping latency minimal? Briefly discuss the protocol phases. What trade-offs arise while achieving this (at least 3)? **[3+3=6]**

Ans.

In UET the specification emphasises “ephemeral connections” that require minimal handshake before data begins flowing, with low state overhead per peer.

- Define a “zero or one-round” start of data: e.g., sender begins with data packet that carries a flag to indicate connection initiation.
- Receiver recognises the first packet and allocates state dynamically; afterwards the protocol uses same identifier for subsequent packets.

CSE587: Networks for AI/ML Systems

Endsem (2 hrs) [Total: 62 points]

Dec 6, 2025 [2 hrs]

- Teardown: idle timer or explicit closing; minimal state retention; garbage-collection of stale connections.
- Scalability: avoid per-peer long-lived state; use shared state pools, or hashed endpoint state.

Trade-offs:

- Reliability: Without handshake you may send to a non-ready receiver; must include buffer-availability signalling or fall-back.
- Ordering / flow-control: If no explicit handshake, you may need sequence numbers to guard reordering or duplication.
- Security: Authentication and key-exchange may typically require handshake; you need to embed credentials or pre-share keys to avoid handshake latency.
- Resource usage: Having minimal state means careful resource reclamation.

Implementation considerations: Tagging of initial packet, fast path hardware support, fallback path for slower start when needed.

Q.3. What does the UALink (Ultra Accelerator Link) and UET (Ultra Ethernet Transport) specification define? How does UALink solve the problem differently from UET? Explain the challenges that the design that employs UAL has to solve vs. the one that deploys UET.

[2+2+2=6]

Ans.

UALink (Ultra Accelerator Link) is a new, open industry standard designed for high-performance, low-latency communication between AI accelerators within a single computing "pod". It is specifically for "scale-up" tasks, creating a high-bandwidth, shared-memory fabric that makes multiple GPUs act as a single, powerful unit.

In contrast, **Ultra Ethernet Transport (UET)** is an enhancement of the standard Ethernet protocol, designed for connecting thousands or millions of nodes across a larger, "scale-out" network. Together, the two standards form a complementary two-tiered structure for massive AI systems.

Problem solved by UET vs. UAL

UALink solves the problem of local, shared memory access for scale-up architectures. For large AI models that exceed a single accelerator's memory, UALink allows a collection of accelerators within a single pod to operate as one logical device. This high-speed, direct memory access is critical for performance-intensive training and inference workloads, where a single OS manages all the accelerators.

UET solves the problem of massive, distributed communication for scale-out architectures. For clusters with thousands of pods, UET standardizes how they communicate efficiently. It improves upon existing Remote Direct Memory Access (RDMA) over Ethernet by optimizing for AI traffic patterns, offering fast connection setup, fine-grained congestion control, and secure communication for distributed workloads.

Design challenges for UALink vs. UET

CSE587: Networks for AI/ML Systems
Endsem (2 hrs) [Total: 62 points]
Dec 6, 2025 [2 hrs]

	UALink (UAL)	Ultra Ethernet Transport (UET)
Hardware	Requires specialized switches built for memory-semantic operations and very low latency, increasing hardware specialization compared to standard Ethernet.	A key goal is to leverage existing standard Ethernet switches, promoting multi-vendor ecosystems and simplifying deployment.
Latency	Extremely low, sub-microsecond latency is required for memory-semantic operations. Design must optimize for wire speed and reduce protocol overhead as much as possible.	Focuses on reducing "tail latency" (the longest delays) and ensuring rapid congestion management and recovery for a broader range of network scenarios.
Memory management	Must provide a unified, coherent memory space for all accelerators within the pod. The design must handle complex memory translation, synchronization, and atomic operations efficiently.	Requires robust protocols for Remote Direct Memory Access (RDMA) to efficiently move bulk data across the network to remote memory without CPU intervention.
Scalability	Must scale effectively from a handful of accelerators up to over a thousand within a confined space. It must manage traffic within a tightly coupled, high-density pod.	Must handle scaling to massive numbers of endpoints (over a million) across vast data center networks. It must manage a large, interconnected fabric reliably.
Interoperability	As a new standard, it needs to ensure interoperability between accelerators and specialized switches from different consortium members to build a healthy ecosystem.	Builds on a decades-old standard, which offers inherent, broad interoperability. The challenge is ensuring seamless coexistence with existing Ethernet traffic and equipment.

Q.4. With fabrics designed to service thousands of endpoints, multi-path traffic, high speeds, and low latencies (for both UET and UALink), diagnosing performance problems (e.g., hot

CSE587: Networks for AI/ML Systems

Endsem (2 hrs) [Total: 62 points]

Dec 6, 2025 [2 hrs]

spots, tail-latency outliers, mis-routed flows) becomes critical. How would you design telemetry (metric measurement) mechanisms into both the transport and link layers to enable this at scale (i.e., what metric and where)? What would you do to avoid significant measurement overheads and ensure high-performance behavior? **[3+3=6]**

Ans.

Telemetry/Instrumentation design:

- In-hardware counters at NICs/switches: packet loss, ECN marks, trimming events, path latency, queue depths.
- Flow tracing: sample flows, record path taken, delays, number of retransmits.
- Distributed logs: endpoints log timestamps of send/receive, sequence numbers, latency per packet.
- Aggregation: Use lightweight streaming telemetry to central collector.

Minimising disturbance:

- Sampling rather than full trace of all packets.
- Offload telemetry tasks to dedicated hardware or low-priority threads so performance paths aren't interfered.
- Use in-band marking (e.g., special extension headers) sparingly.

Q.5. Suppose you want to access a streaming video application with 3 streams with 100 packets per stream. Compare the working of HTTP/1, HTTP/2, and HTTP/3. Use representative figures to explain how HTTP/X works and its challenges. Discuss the advantages and disadvantages of each. **[2*3 = 6]**

Ans.

- **Draw figures** to demonstrate the problems
 - HTTP/1: per-request TCP connection, HOL blocking;
 - HTTP/2: single/persistent TCP connection, HOL blocking within a stream;
 - HTTP/3: no connection since it works on UDP transport; Avoids HOL blocking using parallel streams

Q.6. Suppose you are running a distributed ML workload (training, inference, or serving) over "N" GPUs. Answer the following. **[2+2+1+1=6]**

(a) Which ML workload component runs on the CPU, and what runs on the GPU?

(b) Suppose the ML training workload implements ONLY **data parallelism** with degree equal to 2. How many pods would you require to implement the CPU component? To synchronize weights and gradients, what is the simplest way to connect them in a network?

(c) Suppose we add more GPUs and the data parallelism degree increases to 8, and all GPUs are on the same node/server. How would you implement container networking in the simplest way?

(d) Suppose 8 GPUs are scattered across multiple nodes/servers. How would you implement container networking?

CSE587: Networks for AI/ML Systems

Endsem (2 hrs) [Total: 62 points]

Dec 6, 2025 [2 hrs]

Ans.

(a) CPU Responsibilities: The CPU executes the *control plane* and *input pipeline* of the ML workload. Typical tasks include:

1. Data loading & pre-processing
 - Reading batches from disk / remote store
 - Parsing, tokenization, augmentation, normalization
 - Host-side dataset sharding across data-parallel workers
2. Model orchestration & scheduling
 - Python interpreter, PyTorch/TensorFlow runtime, DDP controller
 - Launching GPU kernels and managing CUDA streams
3. Communication logic
 - NCCL rendezvous, TCP/IP or RDMA endpoint setup
 - Parameter server code (if used)
 - Collective-comms state machine (reduce-scatter, all-reduce orchestration)
4. Checkpointing & logging
 - Saving weights, loading checkpoints
 - Metrics/logging/monitoring

GPU Responsibilities: The GPU executes the *data plane* of ML computation, i.e., all heavy tensor ops:

1. Forward pass
 - Matrix multiplications, convolutions, attention kernels
2. Backward pass
 - Gradient computation
3. Gradient/weight communication kernels
 - GPU-to-GPU NCCL collectives
4. Serving and inference kernels
 - Dense/TensorCore compute, GEMMs, attention, beam search kernels

(b) Pure data parallelism with 2 workers (each with 1 GPU). Each worker has CPU component (Python runtime, dataloader), One GPU, and NCCL or parameter-server client. So 2 workers = 2 pods (implicitly bundled with their GPU worker).

Directly connect the workers/pods using “veth” interface (like a wire)

(c) Connect all 8 pods on the node via a virtual switch or a Linux bridge. Each pod has a veth pair, with one veth interface within the pod and the other within the switch/bridge. The switch connects to the host via the **vnet** interface.

(d) Connect multiple nodes via a gateway that connects the **vnet** interface to the ethernet interface using NAT. One can use CNIs such as Calico and Cilium for large clusters for ease in deployment at the cost of performance.

Q.7. Why do we need both network layer security and transport layer security? Can't we have either one of them? At what layer do IPsec and TLS work? Among the following functions, tell whether IPsec and TLS will support them:

(a) Preventing Network Eavesdropping; (b) Server/Service Authentication; (c) Protecting against a compromise at the host; (d) Securing traffic through L7 proxies/loadbalancers; (e) Securing non-web/legacy traffic **[2+1+3=6]**

CSE587: Networks for AI/ML Systems

Endsem (2 hrs) [Total: 62 points]

Dec 6, 2025 [2 hrs]

Ans.

- Network-layer security and transport-layer security solve different threat models and protect different trust boundaries. Network-layer security (e.g., IPsec, WireGuard, VPNs) protects links between machines or networks. It does not protect data end-to-end between applications.
- Transport-layer security (TLS) protects individual application connections end-to-end, even across untrusted networks, proxies, NATs, middleboxes, CDNs, and load balancers.
- TLS only protects apps that explicitly use it — many protocols don't. Many internal services, legacy applications, storage protocols, and low-level system traffic do not use TLS. IPsec encrypts them transparently.
- IPsec supports (a) and (e)
- TLS supports (a), (b), (c), (d)

Q.8. In a large-scale, distributed LLM training environment (8 nodes x 1 GPU) relying on NCCL for collective communication and RoCE for inter-node data transfer, describe the communication workflow for a single iteration of the All-Reduce gradient synchronization, including the out-of-band control path and the data path execution. **[2+2+2=6]**

Ans. @TAs Check for main points to be covered

The All-Reduce operation is broken into two main stages: a Control Path Setup (slow, managed by CPU/kernel) and the Data Path Transfer (fast, zero-copy, managed by NIC).

Control Path Setup (Out-of-Band Handshake)

This phase establishes the secure, dedicated **Queue Pair (QP)** connection and grants the necessary memory access rights. It is performed once per connection setup.

1. Resource Allocation (CPU/Kernel):

- The NCCL/RDMA stack on both the sender and receiver nodes (driven by the CPU) allocates memory for the **Work Queue Pair (QP)**, consisting of a **Send Queue (SQ)** and a **Receive Queue (RQ)**. A dedicated QP is created for this specific communication flow.
- A **Completion Queue (CQ)** is also allocated to collect status notifications for the QP.

2. Memory Registration (Pinning):

- The CUDA runtime informs the RDMA driver of the **GPU Memory Buffer** containing the gradients that needs to be accessed by the NIC.
- The driver **pins** this GPU memory, creating a **Memory Region (MR)** and generating unique access keys: the **Local Key (L-Key)** for local access and the **Remote Key (R-Key)** for remote access. This is the prerequisite for **zero-copy**.

3. QP Transition & Handshake (Out-of-Band):

- The QP is transitioned through states (INIT → RTR → RTS).

CSE587: Networks for AI/ML Systems

Endsem (2 hrs) [Total: 62 points]

Dec 6, 2025 [2 hrs]

- The sender and receiver exchange necessary connection information (the receiver's **Queue Pair Number (QPN)**, the **R-Key** of the destination memory buffer, and the destination IP/port) over a **standard TCP socket** (the out-of-band channel). This ensures mutual consent and necessary addressing information before the high-speed data path is activated.

Data Path Execution (Zero-Copy Transfer)

Once the QP is in the RTS (Ready to Send) state, the inter-node gradient transfer occurs with minimal latency.

1. Initiation (Posting Work Requests):

- **Sender Side:** NCCL places a **Send Work Request (WR)** into the **Send Queue (SQ)** of the local NIC. The WR specifies the local L-Key (source) and the remote R-Key, QPN, and memory address (destination).
- **Receiver Side:** NCCL must have previously placed a **Receive Work Request (WR)** into the remote NIC's **Receive Queue (RQ)**, pointing to a waiting buffer in the remote GPU memory.

2. Kernel Bypass & Data Transfer:

- The RDMA NIC autonomously processes the WR from the SQ.
- The NIC's **Direct Memory Access (DMA)** engine reads the gradient data **directly from the pinned GPU memory** (bypassing the CPU kernel and caches) over the PCIe bus.
- The data is encapsulated into RoCE packets and transmitted.

3. Remote Write (RDMA Write):

- The remote NIC receives the packets and uses the embedded **QPN** to identify the correct QP/RQ.
- Using the received **R-Key**, the remote NIC's DMA engine performs a **Remote Direct Memory Access (RDMA)** Write operation, depositing the data **directly into the specified buffer in the remote GPU's memory**.

Completion, Notification, and Reliability

Reliability is built into the QP mechanism itself (in this case, Reliable Connected (RC) service type is typically used).

1. Completion Queue (CQ): Upon successful execution of a WR (data sent/data

received), the NIC places a **Completion Queue Entry (CQE)** into the shared **CQ**.

2. Synchronization (Notification):

- NCCL actively **polls** the CQ (reads it repeatedly) to check for CQEs. This is the lowest latency method and critical for minimizing the synchronization overhead in All-Reduce.
- Once the sender reads the CQE for the *send* operation and the receiver reads the CQE for the *receive* operation, both nodes know the cross-node segment of the All-Reduce is complete, and the local GPU can proceed to the final summation.

CSE587: Networks for AI/ML Systems

Endsem (2 hrs) [Total: 62 points]

Dec 6, 2025 [2 hrs]

3. **Atomic Execution:** The use of the QP ensures **ordering and atomicity** of messages. The gradients are guaranteed to arrive in the correct sequence, which is vital for the mathematical correctness of the All-Reduce operation.

Q.9. What is homomorphic encryption? What are the areas in AI where this technology is required? **[1+2=3]**

Ans.

Homomorphic encryption (HE) is a form of encryption that allows computations to be performed directly on encrypted data (ciphertext) without first decrypting it. The result of the computation remains encrypted and, when decrypted by the owner of the private key, yields the same output as if the operation had been performed on the original, unencrypted data.

Homomorphic encryption is essential in AI for enabling Privacy-Preserving Machine Learning (PPML) and secure collaboration on sensitive data, especially when using third-party cloud services.

Examples: Secure Model Training on Sensitive Data, Private AI Inference (Prediction as a Service), and Federated Learning (FL) with Privacy

Q.10. [2+1+2=5]

(a) Compare the protocol stacks of InfiniBand (IB) and RoCE by focusing on protocols adopted by RoCEv2 versus those of IB.

(b) Infiniband assumes a loss-free network, which is not true when RoCE is implemented. How does RoCE resolve this problem? What are the limitations of the proposed solution?

Ans.

(a)

RoCE essentially takes the upper layers of the InfiniBand protocol stack and replaces the proprietary lower layers (link and physical) with standard Ethernet:

Protocol Layer	InfiniBand (IB)	RoCEv1
Transport Layer & Above	InfiniBand (IB): Includes RDMA verbs, Queue Pair (QP) management, and reliable transport mechanisms (e.g., credit-based flow control logic).	Adopts IB Stack.

CSE587: Networks for AI/ML Systems

Endsem (2 hrs) [Total: 62 points]

Dec 6, 2025 [2 hrs]

Network Layer (Layer 3) @TAs: ignore this	InfiniBand: Uses a Global Routing Header (GRH) and a centralized Subnet Manager (SM) for routing.	Ethernet (Layer 2 Only)
Link Layer (Layer 2)	InfiniBand: Proprietary, lossless link layer with credit-based flow control.	Ethernet MAC: Replaces IB's link layer.
Physical Layer (Layer 1)	InfiniBand: Proprietary hardware (HCAs, IB switches, dedicated cables).	Ethernet: Standard Ethernet hardware.

(b) InfiniBand is designed to be lossless, a characteristic that is fundamental for its extremely low-latency, kernel-bypass RDMA operation. Standard Ethernet, by default, is a lossy network that drops packets during congestion.

Since RoCE uses IB's transport layer (which assumes losslessness and lacks heavy retransmission/recovery logic like TCP), the underlying Ethernet network must be made lossless to prevent severe performance degradation. Priority Flow Control (PFC) is implemented to resolve the issue.

However, PFC suffers from several limitations such as Head-of-Line (HOL) Blocking, causes congestion spreading, configuration and management complexity (queue length thresholds).

@TAs: see that the limitations are elaborated with an example or so.

Q.11. Draw a block diagram with the following components and their connections. **[2+1+1+1=5]**

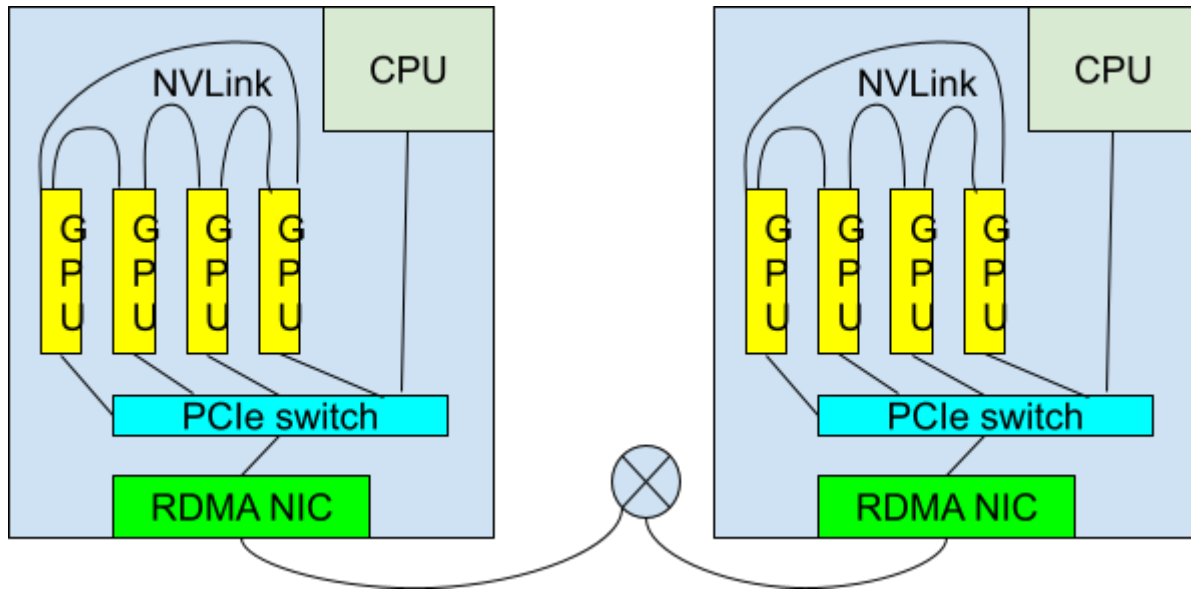
(a) Two server nodes with 4 GPUs each. The GPUs have accelerated connections for intra-host communication and inter-host connections via a switch. Include all the necessary components to set this up.

(b) Show the data flow path between two GPUs within the same server.

(c) Show the data flow between two GPUs across servers.

(d) Show the data flow between the GPU memory and the CPU's DRAM on the same server.

Ans.



**** Show the flows for the questions (b), (c), (d)**

Q.12. List three main challenges that you encountered while executing your course project. If you had to extend your project towards a decent product/tool/research publication, what system components (with brief description) would you want to include? What challenges do you anticipate in doing so? **[1+2+2=5]**

Ans. Subjective
